# The Urban Monitor

VOL. 1, NO. 087

2026-03-30

COVER STORY

# Automatic Object Linkage, with Include Graphs

*In this post I describe an alternative approach to sharing source code elements between multiple build targets.*

*Thomas Young (upcoder)*

It's based on a custom build process I implemented at PathEngine for our C and C++ code-base (but the core ideas could also be relevant to other languages with compilation to object files).

We'll need to enforce some constraints on the way the source code is set up, notably with regards to header file organisation, but can then automatically determine what to include in the link operation for each build target, based on a graph of include relationships extracted from the source files.

The resulting dynamic, fine grained, object-file centric approach to code sharing avoids the need for problematic static library decomposition, and the associated 'false dependencies'.

id="motivation">Motivation

We need to work with a bunch of different build targets, at PathEngine. There's a run-time pathfinding dll, for example, but also a 3D content processing dll, a 3D testbed application, and so on, with a lot of source code shared between these different targets.

The code was originally split into a set of (internal) static libraries, for this purpose. based on things like theme (e.g. 'Geometry.lib') or application layer ('PathEngine_Core.lib', 'PathEngine_Interface.lib'). Different targets could then share common source code by sharing these static libraries.

This worked, as far as it goes, but something didn't feel right.

The libraries weren't particularly modular, for one thing, and just felt like big old bags of object files that didn't really reflect the actual structure of the underlying code.

A bigger problem was that, for any kind of prototyping work, I ended up having to link in a big chunk of the PathEngine source code, even where only some small part of this was really required. Unit testing was also difficult to setup, for a similar reason.

Since throwing out the static libraries and switching to an alternative approach (as described in this post), prototyping and unit testing tasks are a pleasure, with minimal dependencies and short turnaround times, and project setup is more dynamic and does a much better job of reflecting actual code structure.

id="code-sharing">Code sharing

In this previous post , I argued against splitting large projects into static libraries.

The key points in that post were:

static libraries are little more than archived collections of object files

linking object files into static libraries doesn't hide much information, or enforce modularity

grouping objects into static libraries adds 'false dependencies', in that linking with one of the objects then requires the whole library

I ended up glossing over an important point in that post, though, I think.

The thing is, static libraries serve another, more basic, purpose, in providing a way to compile some set of source files just once and then share the result between multiple projects.

Regardless of the problems with static libraries, a lot of people still need to use them, if just for this reason.

This post describes another way to share code between build targets, then, without the false dependency problem associated with static library groupings.

id="disclaimer-some-idealism-here-potentially-difficult-to-apply-in-practice">Disclaimer: some idealism here, potentially difficult to apply in practice

Personally, I'd like to see the approach described here supported by all of the major C++ build tools and IDEs, but this is not the current reality.

A lot of existing build tools and IDEs seem to be built fundamentally around static library decomposition as the basic tool for sharing objects, this is unlikely to change any time soon.

Switching to the new approach at PathEngine required source code changes throughout the code base, and the implementation of a custom build system, and you have to be something of an idealist to take this path, or at least ready to forgo some of the comforts and benefits of mainstream IDEs.

id="starting-with-a-single-project">Starting with a single project

The whole thing really only makes sense where code is shared between multiple projects, but, for simplicity, we'll begin by looking at a single, simple project (an imaginary game application) before extending this to add some additional targets.

We'll start by looking at how compilation dependencies are automated in existing build systems, and then think about how this can be extended, based on certain constraints, to linker dependencies as well.

id="compilation-dependencies">Compilation dependencies

If I make a change to one of the files in a project, and then rebuild, I don't want to wait while all source files in the project get recompiled. The build system should know something about the set of compiled objects potentially affected by the change, and recompile only those objects.

Let's think about how this works.

Consider some source files that make up our game application:

I've added black arrows for each header file included directly by 'game_main.cpp', and then green arrows for other headers brought in by those headers. Taken together, I'll call this the 'include graph' for this source file.

(For clarity, I'll use the term 'source file', in this post, to refer specifically to files like 'game_main.cpp' that get compiled into objects, and just plain 'file' if I want to include 'source files' and headers.)

If any file reachable through the include graph for 'game_main.cpp' gets modified, 'game_main.obj' may be out of date, and should be recompiled.

Files not reachable through this graph (such as 'graphics_util.cpp') have no effect on the compilation of 'game_main.cpp', and these other files can be changed without 'game_main.cpp' needing to be recompiled.

Include graph generation is probably taken care of transparently behind the scenes, by your IDE, but there are tools for extracting this information, and this is something we'll come back and look at in more detail, later on.

id="linker-dependencies">Linker dependencies

What about when we come to link the object files together into a final executable? How does the build system know which object files are required by the link operation?

For our imaginary game application the answer is pretty simple. The source files are there for a reason, they're presumably all needed, and should all be compiled and linked in to the final executable.

(This is how IDEs tend to work, btw.)

But what if these source files were mixed in with a bunch of other source files. Is there some way to automatically determine a set of objects to link?

id="inferring-link-requirements-from-included-headers">Inferring link requirements from included headers

Consider one of our headers, 'draw_player.h'. This might look something like the following:

class IRenderer ; class CPlayerState ;

void DrawPlayer ( const IRenderer & render , const CPlayerState & playerState );

This header tells the compiler how DrawPlayer() should be called, with the actual implementation of DrawPlayer() (hopefully) provided in 'draw_player.cpp'.

The information in the header is sufficient for the compiler to generate calling code (setting up the stack set with relevant parameters and making the call), but the actual compiled code for the implementation of function, and the

location of the call target, will need to be resolved at link time, by linking with ‘draw_player.obj’.

The same is true for headers with class method declarations, as in the following:

class CPlayerState { //... load of private stuff

public :

CPlayerState ( const char * name );

int GetHealth () const ;

//... load of other stuff }

Again, both the constructor and the GetHealth() method shown here need to be implemented in ‘player_state.cpp’, and resolved at link time by linking with ‘player_state.obj’.

In each case there’s a matched pair of files with one header and one source file (‘draw_player.h’/‘draw_player.cpp’, ‘player_state.h’/‘player_state.cpp’), and using stuff in the header creates a link dependency on the corresponding cpp file.

If you look at other C++ features, such as static class data members, you’ll see that this works exactly the same way, and, the way I see it, this is really just the way headers are supposed to be used , in C and C++.

Taking one step back, to use stuff in the header you have to include that header. Also, source files should ideally only include headers that they actually use .

So what we’re going to do is, wherever a header has a matching source file, and something includes the header, infer a linkage requirement to that source file.

We’ll come back and look at the implications of this later on, but first let’s see what this buys us.

id=“adding-linkage-requirements-to-the-dependencies-graph”>Adding linkage requirements to the dependencies graph

Adding our new, inferred, linkage requirements in to the dependency graph for ‘game_main.cpp’ gives us the following:

The idea here is that all the cpp files reachable through this graph are required in order to satisfy potential link requirements in ‘game_main.cpp’.

id=“and-again-recursively”>And again, recursively..

Linking with this set of objects is not sufficient, however. Each object brought in to the link has its own dependencies to be satisfied, as well.

To satisfy all the link dependencies for our project, we have to generate include graphs for each new object, recursively, expanding our linkage dependency graph as we go.

We could expand ‘world_update.cpp’ next, for example, with the expansion looking like this:

As before, black arrows and green arrows indicate the include graph for the object being expanded, and red arrows show inferred linkage. I’ve added yellow arrows to show the linkage dependency graph built up so far.

Some of the dependencies of ‘world_update.cpp’ are already in our linkage graph, and no further action is required for these source files, but ‘enemy_ai.cpp’ is new, and we’ll add this.

I won’t show all of the source file expansions, but if we continue expanding new source files we’ll eventually satisfy all linkage requirements, at which point we have a complete set of source files to compile and link into an application.

In our case the last required source file, ‘graphics_util.cpp’ gets added (for example) when expanding the subgraph for ‘draw_player.cpp’:

id=“what-does-this-give-us”>What does this give us?

This may seem like a lot of messing around for our simple game application, where linkage requirements are obvious, but starts to make more sense if we add some additional targets.

Let’s make this into a networked game, then. We’ll split the code into two applications, one for the game server, and another for the game client.

Each application is defined by a single root source file, and expanding linkage requirements our from root source files gives us the set of source files actually needed to satisfy linkage requirements for each application.

For the server, this works out as follows:

For simplicity, I’ve removed the ‘base_types’ header (which doesn’t actually have any effect on linkage requirements), and combined source/header pairs into single nodes (effectively collapsing those red arrows).

A new source file, ‘networking.cpp’, is used to pass serialised information about world state over the network. The server just updates the world, and runs the enemy AI, but doesn’t handle player input or rendering.

The linkage dependencies for our client application are generated from the same source tree, and look like this:

The first advantage of the PathEngine build system, then, is automatic generation of the set of objects to link for each build target.

This avoids work setting up and maintaining lists of objects in application projects and shared libraries, and dependencies between those projects.

Eliminating manual project configuration elements eliminates the possibility of human error in the way these project configuration elements are setup.

Automated dependencies also makes the source code more dynamic and easier to refactor, since we can change relationships between source code elements directly in the source code, without worrying about the need to wrestle with project setup (even for changes with significant implications for dependencies).

That's all good stuff, but there are other advantages of this set up, which brings us to the second key feature of the PathEngine custom build system, object sharing.

id="shared-object-files">Shared object files

We can see from the diagrams above that some of the objects ('networking.cpp', 'player_state.cpp' and 'world_state.cpp') are used by both server and client.

With our custom build system, the corresponding object files ('networking.obj', 'player_state.obj' and 'world_state.obj') are then shared by the two applications. If I build the server, and then the client, these object files are generated only once, and then get reused.

There are some implications of this setup. You can't set different compiler settings for a given source file for different targets, and this means you can't use constructs like the following:

#ifdef BUILDING_CLIENT //... some logic #endif

#ifdef BUILDING_SERVER //... some other logic #endif

This is the same situation as for objects in a static library, however, and you can kind of think of each object file as being its own mini static library.

In practice I prefer to avoid this kind of construct. I think source files that 'compile only one way' are easier to understand, and in return we get the significant benefit (as with code sharing through static libraries) of avoiding recompilation, hence faster build times.

id="unit-tests">Unit tests

I mentioned unit testing as one of the benefits of the new approach. Lets see how this looks for our game source code.

Well, each unit test is essentially just another 'root source file' for which a set of linker dependencies can be determined.

id="isolating-individual-unit-tests">Isolating individual unit tests

When the source file under test is mostly independent from other source files, we get a nice minimal set of objects to build:

Being able to build and run just a single test, with low turnaround times, is great for iterating on initial test set up, when adding regression, and when debugging the test to fix a complicated issue, and it's great to be able to run individual unit tests when other bits of source code are broken for some reason.

A minimal working set, with less code linked into a test binary, also means less chance of side effects from other code and less mental load, with less things potentially needing to be considered.

id="grouping-unit-test-together">Grouping unit test together

A lot of the time you probably just want to verify that the tests all still pass, and building an individual exe for each unit test would then be overkill.

The trick, in this case, is to start the dependency analysis with more than one root source file and build multiple tests into a single executable.

The PathEngine build system does this for sets of tests matching a regular expression, omitting tests which have already passed and are unaffected by source code changes, (but full details about how to set this up are beyond the scope of this blog post).

id="higher-level-testing">Higher level testing

We can apply the same principle, also, to less 'independent' source files, and end up with a kind of mid-level test:

id="working-with-the-same-compiled-result">Working with the same compiled result

In each of these testing use-cases, compiled objects are shared with the main application, and we're testing exactly the same compiled code as gets linked into our production application.

This avoids situations, notably, where an issue is caused by a compiler bug, or only repeats when code is compiled a certain way, but doesn't repeat when the code is compiled differently for testing.

id="source-file-organisation-is-often-orthogonal-to-dependency">Source file organisation is often orthogonal to dependency

When I set up static libs for PathEngine based on theme or application layer, this turned out to be a bad way to manage dependencies, but that doesn't mean that these are inherently bad ways to organise source code .

Let's take one more look at our networked game source code:

id="grouping-by-layer">Grouping by layer

A couple of source files are different in that they don't know anything about the specific application domain of our game, and I'm going to call this group of files 'the platform layer'. These files can go in a separate, suitably named, top-level directory from the rest of the source code.

I'd also like to add a constraint that code in the platform layer should never include or call into lower 'application' layers.

This seems like it can be a useful way to group source files, but that doesn't mean that the code should also be clumped together into a single dependency unit. Forcing the game server to link with 'graphics_util.h' certainly seems like a bad idea. (Maybe this adds a dependency on DirectX, and the relevant runtime is not installed on the server machine..)

id="grouping-by-theme">Grouping by theme

Another two files both share the common feature of drawing something . Grouping these files into the same 'Rendering' subdirectory also seems like a good idea.

We may turn out to need other things rendered, and so other similarly themed source files are likely to get added, as our application grows. It will be nice to able to locate these files quickly, and to get an idea about what kinds of different things our source code is currently able to render, but there's no reason to think that code that wants to draw one thing (e.g. the player) will always also want to draw everything else.

Separating these source file groupings out from project dependency structure also has the effect of making the grouping more dynamic. If one of these groupings turns out not to make so much sense later on, it's great to be able to change these groupings without subsequently wrestling with dependency implications to make everything work once again.

id="inferring-link-requirements-what-could-go-wrong">Inferring link requirements, what could go wrong?

I've hopefully shown, at this point, that automatic object linkage is nice to have , but our implementation of this is based on some assumptions about the source code.

Specifically, we're assuming that source files and headers are always set up as matching pairs, that code does not 'bypass' headers and declare linkage directly, that headers actually do contain elements that require linkage to the corresponding source file, and that source code that includes the headers actually does use these elements from the headers.

None of this is guaranteed by the standard. What could go wrong?

Well, just two things, essentially:

Linking not enough objects

Linking too many objects

id="bypassing-a-header">Bypassing a header

Instead of including 'draw_player.h', someone might do the following:

void DrawPlayer ( const IRenderer & render , const CPlayerState & playerState );

int main ( int argc , char * argv []) { //... set up renderer //... set up world and player state

//... other stuff

DrawPlayer ( renderer , playerState );

//... other stuff }

What happens?

Nothing includes "draw_player.h", we fail to detect the linkage requirement, "draw_player.cpp" is not compiled or linked in, and we get a link error as follows:

game_main.obj : error LNK2019: unresolved external symbol "void __cdecl DrawPlayer(class IRenderer const &,class CPlayerState const &)" (? DrawPlayer@@YAXAEBVIRenderer@@AEBVCPlayerState@@@Z) referenced in function main

Well, first of all, I don't want anyone to write code like this. There's a reason for the header. Bypassing the header makes the code brittle with respect to changes to the DrawPlayer() function signature, but also makes it harder to identify code that uses rendering functions (by searching for include lines), and this is a code quality issue that I'm happy to see identified by an error.

But the point is, even with code like this, the result is far from catastrophic. It's the same error as if you forget to add a new cpp file to a Visual Studio project, with standard project, but less likely to happen, and with the same kind of direct line back from error to cause.

id="global-linkage-requirements">Global linkage requirements

Another way we might miss linkage requirements is due to the kind of 'global' linkage requirement implied by something like the following:

extern "C" { int PathEngine_HandleAssertion ( const char * , int32_t , const char * ); }

#define assertR(expr) do{static int on = true;if(on && !(expr)) on = PathEngine_HandleAssertion(__FILE__,__LINE__,#expr);} while(0)

#ifdef ASSERTIONS_ON #define assertD(expr) assertR(expr) #else #define assertD(expr) do{}while(0) #endif

In this case, there is no single corresponding "Assert.cpp". Instead, different types of application are expected to provide suitable implementations of PathEngine_HandleAssertion(), depending on the desired behaviour. (In some cases a message box should pop up. In others, assertions should be printed to console output, or logged. Sometimes it should be possible to turn assertions off, and so on.)

But what if I want to share common assertion handling strategies between applications?

I can set up a source file "MessageBoxAssert.cpp", for example, but no header is required for this (with PathEngine_HandleAssertion already declared in "Assert.h"). Because there is no matching header pair, nothing tells the build system that this source file needs to be compiled, or that "MessageBoxAssert.obj" needs to be included in the link.

There are perhaps better ways to handle this kind of situation, but what we do currently in PathEngine, is go ahead and add an empty "MessageBoxAssert.h", anyway. This (empty) header file can be included from the relevant application root source file just to indicate that linkage to "MessageBoxAssert.cpp" is desired.

id="linking-objects-that-arent-required">Linking objects that aren't required

Consider the following class:

class CSerialiser ;

class CPosition { int x ; int y ; public : CPosition ( int x , int y ) : x ( x ), y ( y ) { }

int getX () const { return x ; } int getY () const { return y ; }

void translateBy ( int dx , int dy ) { x += dx ; y += dy ; }

void serialise ( CSerialiser & s ) const ; };

Not a hugely useful class, but the point is that nearly everything here is implemented inline in the class header.

There is a matching "position.cpp", however, containing just the definition of CPosition::serialise().

Perhaps I have some code that needs to do some stuff with positions, but doesn't need to serialise them.

Due to the way we infer linkage dependencies, "position.cpp" will get pulled in to the link, as well as "serialiser.cpp" (and probably whole a bunch of other stuff) even if none of this is actually used.

Note that I haven't seen this issue in PathEngine, in practice , and this is really just a theoretical objection to inferring linkage requirements the way we do.

I guess the point is that, although the method we are using for inferring linkage dependencies is more fine grained than and relationships between static libraries, in some cases it may not be fine grained enough.

And I guess the answer is then that, sure, the approach is not perfect but it's nevertheless a significant improvement on what we had before, and also turns out to be a good fit for a bunch of situations, in practice.

Perhaps it's possible to go further and analyse each source file to determine whether there actually are declarations brought in from headers, without associated definitions, but that would be a big increase in complexity, and I don't think this is necessary.

It's probably better to change the code in some way, so that the relevant dependency is explicit in the source code include relationships.

For this position class, one possibility would be to pull the serialisation operation out of the class and express this in terms of externally visible methods, and another would be to hide the implementation details of the serialiser behind a virtual interface.

Setting up a tool to answer queries about why specific objects are included in a link operation, by reporting the relevant linkage chain, can help a lot with this.

id="extracting-includes-from-source-files">Extracting includes from source files

Let's come back and take a look, finally, at the process of extracting the relevant include graph information from our source files.

The simplest way to do this, these days, is probably just to use a compiler option that tells your compiler to do this.

In the case of gcc (and clang) you can use the -M (or -MM) compiler option (see this page, in the docs ). For the Microsoft Visual C++ compiler there's /showIncludes .

Using the compiler to do this makes sense because the process of extracting includes is a subset of the compile task itself.

Notably, for each source file analysed in this way, each included header needs to be loaded in from disk, and examined in the specific preprocessor context at that point in the compile.

And this means that, when iterating over the set of all source files, certain commonly included headers may end up being loaded and examined many times over.

To illustrate exactly why this is necessary, consider the following (made up) example:

#include “base_stuff.h” #ifdef MESSAGE_BOXES #include “message_box_errors.h” #else #include “console_errors.h” #endif

//.. bunch of other stuff

This potentially results in a different set of includes, depending on the state of the REPORT_ERRORS define, which may be different for different source files, depending or how it is set in the source file itself, and whether it was set or unset by previously included header files.

If it wasn’t possible for preprocessor conditionals to affect include directives, we could analyse each header individually, and re-use the results wherever the header is included.

But these kinds of preprocessor conditionals are also pretty confusing, in my opinion, and make the code harder to work with.

What we do in PathEngine, then, is straight-out disallow conditional compilation around include directives. The mechanism for this is pretty straightforward. Include directives are only allowed right at the top of each header, with no other preprocessor statements allowed before or between the include directives.

This might seem like quite a significant source code constraint. Conditional include directives sometimes seem like they are indispensable, particularly when you are building for a lot of different platforms.

In my experience, however, we don’t have to set things up this way. Preprocessor conditionals like this should usually be set consistently across builds, and can then be replaced by the alternative mechanism of include directory search order:

#include “base_stuff.h” /\/ the following might include either /\/ “message_box/errors.h” /\/ or /\/ “console/errors.h” /\/ depending on include directory search order #include “errors.h” #endif

With this constraint in place, the resulting ‘per header’ approach to includes analysis gives us two very significant speedups:

Each header is loaded and processed only once, so fewer files are loaded and processed in total (potentially a change from O(n^2) to O(n), I think, depending on include graph branching factor).

After one header is edited, only that header needs to be reprocessed, and no others (as opposed to all dependent files).

id=“interacting-with-ides”>Interacting with IDEs

For PathEngine, switching to the approach I have described meant implementing a custom build system, but that doesn’t mean we have to go right back to the stone age, or implement our own IDE from scratch.

There are a couple of ways our custom build system can interact with modern IDEs.

id=“external-build-projects”>External build projects

Most IDEs offer some kind of ‘external build’ project setup where you can specify an arbitrary build command, and have build output directed into the standard build output window.

It’s usually straightforward from there to set up filtering for errors, so that ‘go to next error’ functionality can be used, and to run and debug your build result.

After that this approach can offer a lot of the benefits of the custom build system, such as object file sharing, implicit build targets, and unit testing, but at the expensive of features like intellisense, which depend on the IDE understanding exactly how to build your code.

(I’ve implemented a libClang based ‘lookup reference’ feature for an external project setup, and found that I can be quite productive with this kind of minimal ‘intellisense’, but it’s a far cry from all the bells and whistles of a something like Visual Studio, and certainly not for everyone.)

id=“generated-project-files”>Generated project files

The second possibility is to make your build system spit out standard project files for individual targets, for your IDE, when required.

Actually, the main thing we want from the build system is the list of source files to be included for the relevant target, and project files can then generated from templates and filled out with these source files.

The project won’t update dynamically to reflect changes to include relationships, but we can get pretty close by triggering a script to regenerate the projects manually, when necessary. (Visual Studio actually handles background changes to projects you are working on quite nicely. A message box pops up asking if you want to reload the projects, and the reload process all seems to be fairly robust.)

The benefit of this kind of setup is that you can take advantage of all those IDE bells and whistles (intellisense, detailed code highlighting, and so on). The disadvantages are that each project file generated in this way can only build one target, and object files are no longer shared between projects.

id=“juggling-the-two-approaches”>Juggling the two approaches

Between the two approaches it's possible to do pretty much everything you can do with a modern IDE.

Sometimes I'll switch between the two approaches depending on what platform I'm working on, and what exactly I want to do. If I'm prototyping or unit testing, I prefer to use external build projects (and the full power of our custom build system). If I'm working on a larger application, however, on Windows, a generated Visual Studio project can be a better way to go.

Juggling two types of project files is not ideal, however, and it would be great to combine the advantages of the two approaches into a single integrated setup.

id="summing-up">Summing up

With some assumptions about header organisation, we can infer the set of objects required to build an application from just one 'root' source file, and the include relationships implicit in the source code.

The result is a very dynamic approach to project management, which eliminates the need for a lot of manual project configuration, with shared object files, and finer grained dependencies than standard static library decomposition.

We had to add source code constraints to make this work but this works out well in practice and these are arguably good constraints to apply on your source code, anyway.

It's an idealistic approach to project setup, and tricky to reconcile with project configuration options in popular IDEs, but perhaps this is how IDEs should work in the future?

---

{

## FEATURES

# How to preload Rails scopes

*Justin Weiss*

This article is also available in Korean , thanks to Soonsang Hong!

Rails' scopes make it easy to find the records you want:

app/models/review.rb class Review { where ( "rating > 3.0" ) } end

irb(main):001:0> Restaurant . first . reviews . positive . count Restaurant Load (0.4ms) SELECT `restaurants`.* FROM `restaurants` ORDER BY `restaurants`.`id` ASC LIMIT 1 (0.6ms) SELECT COUNT(*) FROM `reviews` WHERE `reviews`.`restaurant_id` = 1 AND (rating > 3.0) => 5

But if you're not careful with them, you'll seriously hurt your app's performance.

Why? You can't really preload a scope. So if you tried to show a few restaurants with their positive reviews:

irb(main):001:0> restauraunts = Restaurant . first ( 5 )

irb(main):002:0> restauraunts . map do | restaurant |

irb(main):003:1* " #{ restaurant . name } : #{ restaurant . reviews . positive . length } positive reviews."

irb(main):004:1> end

Review Load (0.6ms) SELECT `reviews`.* FROM `reviews` WHERE `reviews`.`restaurant_id` = 1 AND (rating > 3.0)

Review Load (0.5ms) SELECT `reviews`.* FROM `reviews` WHERE `reviews`.`restaurant_id` = 2 AND (rating > 3.0)

Review Load (0.7ms) SELECT `reviews`.* FROM `reviews` WHERE `reviews`.`restaurant_id` = 3 AND (rating > 3.0)

Review Load (0.7ms) SELECT `reviews`.* FROM `reviews` WHERE `reviews`.`restaurant_id` = 4 AND (rating > 3.0)

Review Load (0.7ms) SELECT `reviews`.* FROM `reviews` WHERE `reviews`.`restaurant_id` = 5 AND (rating > 3.0)

=> [ "Judd's Pub: 5 positive reviews." , "Felix's Nightclub: 6 positive reviews." , "Mabel's Burrito Shack: 7 positive reviews." , "Kendall's Burrito Shack: 2 positive reviews." , "Elisabeth's Deli: 15 positive reviews." ]

Yep, that's an N+1 query . The biggest cause of slow Rails apps.

You can fix this pretty easily, though, if you think about the relationship in a different way.

id="convert-scopes-to-associations">Convert scopes to associations

When you use the Rails association methods, like belongs_to and has_many , your model usually looks like this:

app/models/restaurant.rb class Restaurant { where ( "rating > 3.0" ) }, class_name: "Review" end

irb(main):001:0> Restaurant . first . positive_reviews . count Restaurant Load (0.2ms) SELECT `restaurants`.* FROM `restaurants` ORDER BY `restaurants`.`id` ASC LIMIT 1 (0.4ms) SELECT COUNT(*) FROM `reviews` WHERE `reviews`.`restaurant_id` = 1 AND (rating > 3.0) => 5

Now, you can preload your new association with includes :

irb(main):001:0> restauraunts = Restaurant . includes ( :positive_reviews ). first ( 5 ) Restaurant Load (0.3ms) SELECT `restaurants`.* FROM `restaurants` ORDER BY `restaurants`.`id` ASC LIMIT 5 Review Load (1.2ms) SELECT `reviews`.* FROM `reviews` WHERE (rating > 3.0) AND `reviews`.`restaurant_id` IN (1, 2, 3, 4, 5) irb(main):002:0> restauraunts . map do | restaurant | irb(main):003:1* " #{ restaurant . name } : #{ restaurant . positive_reviews . length } positive reviews." irb(main):004:1> end => [ "Judd's Pub: 5 positive reviews." , "Felix's Nightclub: 6 positive reviews." , "Mabel's Burrito Shack: 7 positive reviews." , "Kendall's Burrito Shack: 2 positive reviews." , "Elisabeth's Deli: 15 positive reviews." ]

Instead of 6 SQL calls, we only did two.

(Using class_name , you can have multiple associations to the same object. This comes in handy pretty often.)

id="what-about-duplication">What about duplication?

There still might be a problem here. The where("rating > 3.0") is now on your Restaurant class. If you later changed positive reviews to rating > 3.5 , you'd have to update it twice!

It gets worse: If you also wanted to grab all the positive reviews a person has ever left, you'd have to duplicate that scope over on the User class, too:

app/models/user.rb class User { where ( "rating > 3.0" ) }, class_name: "Review" end

It's not very DRY .

There's an easy way around this, though. Inside of where , you can use the positive scope you added to the Review class:

app/models/restaurant.rb class Restaurant { positive }, class_name: "Review" end

That way, the idea of what makes a review a positive review is still only in one place.

Scopes are great. In the right place, they can make querying your data easy and fun. But if you want to avoid N+1 queries, you have to be careful with them.

}

{

# Here's how to live: Commit.

*Derek Sivers*

If you've ever been confused or distracted, with too many options...

If you don't finish what you start...

If you're not with a person you love...

... then you've felt the problem.

The problem is a lack of commitment.

You've been looking for the best person, place, or career.

But seeking the best is the problem.

No choice is inherently the best.

What makes something the best choice?

You.

You make it the best through your commitment to it.

Your dedication and actions make any choice great.

This is a life-changing epiphany.

You can stop seeking the best option.

Pick one and irreversibly commit.

Then it becomes the best choice for you.

Voilà.

When a decision is irreversible, you feel better about it.

When you're stuck with something, you find what's good about it.

When you can't change your situation, you change your attitude towards it.

So remove the option to change your mind.

You think you want more choice and more options.

But when you have unlimited choice, you feel worse.

When you keep all options open, you're conflicted and miserable.

Your thoughts are divided.

Your power is diluted.

Your time is thinly spread.

Indecision keeps you shallow.

Get the deeper pleasure of diving into one choice.

The English word "decide" comes from Latin "to cut off".

Choose one and cut off other options.

To go one direction means you're not going other directions.

When you commit to one outcome, you're united and sharply focused.

When you sacrifice your alternate selves, your remaining self has amazing power.

Ignore other aspects of your life.

Let go of every unnecessary obligation.

Each one seems small, but together, they'll drain your soul.

Focus your attention on the few things you're committed to, and nothing else.

When our ancestors shifted from nomadic hunter-gatherers to settled land developers, human development boomed. We made massive advances when we stopped moving, and committed to one place.

Choose your home.

Stay there for good.

Get to know everything about it.

Even if you've lived there for years, hire a local expert and learn even more about the history, architecture, and areas you haven't explored.

Find a community of like-minded people.

Don't waste your energy fighting norms.

Trust helps your happiness more than income or health.

Invite your neighbors over for a meal.

Make friends.

Make the effort.

Borrow and lend.

Trust and show you can be trusted.

Let them know they can lean on you because you're here to stay.

They'll reciprocate.

You'll be an inseparable part of your community.

The good relationships you build will build you.

}

{

# Here's how to live: Reinvent yourself regularly.

*Derek Sivers*

People say everything is connected.

They're wrong.

Everything is disconnected.

There is no line between moments in time.

Something happened.

Something else happened.

People love stories, so they connect two events, calling them cause and effect.

But the connection is fiction.

It's a hard fiction to escape.

"My parents did that, so that's why I did this."

No.

Those two events are not connected.

There is no line between moments in time.

Same with definitions.

"I'm an introvert, so that's why I can't."

No.

Definitions are not reasons.

Definitions are just your old responses to past situations.

What you call your personality is just a past tendency.

New situations need a new response.

Are you more emotional or intellectual?

Early bird or night owl?

Liberal or conservative?

No.

Disagree with the question.

You aren't supposed to be easy to explain.

Putting a label on a person is like putting a label on the water in a river.

It's ignoring the flow of time.

Your identity.

Your meanings.

Your trauma.

They're all based on the core idea that you're in a continuum, living a story.

But there is no line between moments in time.

There is no story.

There is no plot.

Should you try to be consistent with your past self?

Should a newspaper try to be consistent with past news?

You're an ongoing event — a daily improvisation — responding to the situation of the moment.

Your past is not your future.

Whatever happened before has nothing at all to do with what happens next.

There is no consistency.

Nothing is congruent.

Never believe a story.

You've changed so much over time.

Your past self is as different from your current self as you are from other people.

Your past self needs to step down, like a previous president, to let the new you run the show.

Doing what you've always done is bad for your brain.

If you don't change, you'll age faster and get stuck.

The way to live is to regularly reinvent yourself.

Every year or two, change your job and move somewhere new.

Change the way you eat, look, and talk.

Change your preferences, opinions, and usual responses.

Try the opposite of before.

Disconnect from your past.

Cut all common threads.

Keep nothing permanent.

No tattoos.

Remain a clean slate.

}

{

# Life is _____

*Derek Sivers*

I was at a workshop, and right before dinner, the teacher wrote this on the whiteboard:

LIFE IS _____

He told us to think about what goes in the blank. He said that after dinner, he'd reveal the meaning of life.

At dinner, I was at a table with seven other people, each arguing about what should go in that blank. One said life is learning. One said life is memory, since if you can't remember your life, it's like it never happened. One said life is love — the most powerful emotion. One said life is giving. One nouveau Buddhist said life is suffering, repeating his recent lessons. One said life is choice, since our choices shape our life. One said life is time, since life is what we call the time between when we're born and when we die.

Each was arguing that their answer was definitely the right one. I'm usually talkative, but I stayed quiet and just listened. Because there were different valid perspectives, it seemed clear that none of these could be the answer.

Then I thought maybe there is no answer — there is no built-in meaning. Maybe life is like a blank canvas for everyone to project their own meaning into.

Oh! Maybe that's why the teacher wrote: "LIFE IS _____". Maybe that's not a question! Maybe "_____" is the answer. Ooooh that's good. I like that a lot.

After dinner, yeah, my hunch was right — that's what the teacher intended. He pointed up and asked, "What's the meaning of this ceiling?" Someone said, "It provides shelter." Someone else said, "Safety. Structure." The teacher said, "Those are your meanings. The ceiling itself has no meaning. It's just a ceiling."

He asked everyone, "What does it mean that you're here today?" Someone said, "It means I'm trying to improve myself." Someone else said, "It means I'm committed." The teacher said, "Those are your meanings. Your presence here today has no inherent meaning."

Then he asked, "So what's the meaning of life?" This time people's answers were emphatic, each arguing for their favorite meaning. The teacher said, "Those are your meanings. Life itself has no meaning." Now people were upset, saying this whole workshop was a scam and they want their money back since they expected an answer.

But I like that "_____" answer a lot. Not just for the meaning of life, but for everything.

You love travelling. What does it mean? You must be running away from something? You're privileged? You're a curious soul, searching for answers? Nah.

Nothing has inherent meaning. Whatever meaning you project into it is your own.

You were just thinking of your long-lost friend this morning, and then they contacted you for the first time in years. What does it mean? Our psychic connections bind us? Our souls are in sync? The universe is sending out energy waves that we can feel? I mean, if you like that idea, why not? If that makes life feel more special, more magical... If that makes you curious about the unseen forces all around us... If that makes you marvel and wonder, then maybe that meaning works for you. Great. Give that event that meaning.

That's coming from you.

Though maybe you need to believe it's true to feel its magic power.

Meanings can help you feel your life is important, with a narrative and purpose. Meanings can help you make peace with events out of your control. Meanings can give you a reason to persist in difficult times.

But they're internal, not external.

They're yours, not others'.

Me? I like the "_____". I like the blank canvas. I love that nothing, in itself, has built-in meaning. I love the creative power of choosing my own. Meanings are useful, not true.

}

{

# Hegseth Makes Troops Prove “Sincerely Held” Faith in Latest Beard Crackdown

*Noah Hurowitz · The Intercept*

The latest edict from beard-obsessed Secretary of War Pete Hegseth adds strict new regulations to his crusade on facial hair, which rights groups have characterized as an attack on troops’ civil liberties.

In a March 11 memo, Hegseth, who has made grooming and appearances a central focus in his time at the helm of the U. S. military, raised the bar to qualify for a religious exemption to his blanket ban on beards. The guidelines lay out a strict new process by which service members may apply for a religious exemption and subject those who’ve already received one to a reevaluation, arguing they need to ensure their religious beliefs are “sincerely held” and have a genuine conflict with the grooming standards.

Service members who have spoken against Hegseth’s focus on grooming standards say his restrictions on beards are exclusionary to people from religious communities that require adherents to follow specific tenets of faith around beards, hair, and other grooming matters.

Sikhs, for example, who have served in the U. S. military since at least World War I, are required by their faith not to cut the hair on their head, to keep a beard, and to wrap their long hair in a turban. Members of many schools of Muslim tradition likewise have rules around beards and hair length.

A Sikh advocacy group derided the new requirements as “completely unnecessary.”

“Sikhs and other service members of faith already earned their accommodations, under policies and processes established under both the Obama and first Trump Administrations,” the Sikh Coalition said in a statement. “If there are accommodations that the Department of Defense feels are not sincere, they could have chosen to pursue those cases with a process that doesn’t force every single soldier, sailor, airman, guardian, and Marine with an accommodation through more paperwork and bureaucracy.”

The Department of War did not respond to a request for comment.

Hegseth introduced the new guidelines as the military increasingly embraces overt Christianity and Christian nationalism , including an ideological turn on the Air Force Academy’s oversight board and the presentation of the war on Iran as part of “ God’s divine plan .”

The changes come months after Hegseth declared war on “beardos” in a combative speech in September.

“If you want a beard, you can join Special Forces. If not, then shave,” Hegseth said at the time.

In a November letter to Hegseth, four senators — Gary Peters, D-Mich.; Elizabeth Warren, D-Mass.; Tim Kaine, D-Va.; and Kirsten Gillibrand, D-N. Y. — warned that an overly strict grooming standard could force religious service members from the ranks and ultimately harm the military’s primary mission of national security.

“This will happen either by forcing out servicemembers with accommodations earned through carefully following their branch’s established processes or signaling to members of these religious communities that their contributions are not needed in the world’s greatest fighting force,” the senators wrote. “At a time when readiness and retention remain urgent concerns, such a move would be ill-advised.”

Federal courts have repeatedly ruled in favor of service members’ rights to observe tenets of faith while in the military, limiting Hegseth’s ability to put in place an outright ban on any facial hair. He has opted instead to tighten the screws on anyone wishing to get an exemption.

Courts have generally required the military to accommodate sincerely held religious beliefs unless it can demonstrate a compelling operational need.

Under the new rules, anyone applying for an exemption — or facing reevaluation under the new guidelines — must submit a sworn statement affirming their religious beliefs, a statement detailing those beliefs, a statement explaining how the grooming standard would conflict with those beliefs, and supporting evidence backing up their “sincerely held” beliefs. Additionally, anyone applying for an exemption must receive from their unit commander a written assessment of the applicant’s sincerity of belief.

The policy also places commanders in the position of evaluating the sincerity of a service member’s religious beliefs. False statements could expose service members to disciplinary action under the Uniform Code of Military Justice.

The post Hegseth Makes Troops Prove “Sincerely Held” Faith in Latest Beard Crackdown appeared first on The Intercept .

}

standard-article( title: [The Bitter History of Chocolate], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [What's better than holiday hot chocolate? If just thinking about it makes you feel all warm and fuzzy, well – that's by design. Chocolate's big history sweeps across the globe, and today we're going on that journey: from the pre–Columbus Americas, to an early 20th century reporter's hunch about what cocoa production really takes, to a 21st century medical student's story about his childhood on a farm that produces those holiday treats.], [Guests:

Carla Martin, lecturer in African and African American Studies at Harvard University and President of the Board of the Institute for Cacao and Chocolate Research], [Catherine Higgs , professor of history at the University of British Columbia in Canada], [Shadrack Frimpong , founder of Cocoa360], [We've got a favor to ask: We know there are a lot of great NPR shows out there.. but we all know who's the best. NPR is celebrating the best podcasts of the year, and YOU get to crown the winner of the People's Choice Award. Vote for Throughline at npr.org/peopleschoice. May the best pod win!], [To access bonus episodes and listen to Throughline sponsor-free, subscribe to Throughline+ via Apple Podcasts or at plus.npr.org/throughline .], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 233, edited-for-length: false, debug-mode: false, )

standard-article( title: [The Creeping Coup], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Sudan has been at the center of a deadly and brutal war for over a year. It's the site of the world's largest hunger crisis, and the world's largest displacement crisis.], [On the surface, it's a story about two warring generals vying for power – the latest in a long cycle of power struggles that have plagued Sudan for decades. But it's also a story about the U. S. war on terror, Russia's war in Ukraine, and China's global rise.], [Today on the show, we turn back the clock more than a century to untangle the complex web that put Sudan on the path to war.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 145, edited-for-length: false, debug-mode: false, )

brief-group(( [

**Patrick McKenzie (patio11), patio11 (Patrick McKenzie):** I joined Stripe to work on building financial infrastructure for the Internet. Time flies. Here is what I've learned in the first four years.

], [

**Throughline (NPR), Throughline (NPR):** Mitch McConnell has been described as "opaque," "drab," and even "dull." He is one of the least popular - and most polarizing - politicians in the country. So how did he win eight consecutive elections? And what does it tell us about how he operates? This week, we share an episode we loved from Embedded that traces McConnell's political history.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Throughline (NPR), Throughline (NPR):** A record number of Americans have died from opioid overdoses in recent years. But how did we get here? And is this the first time Americans have faced this crisis? The short answer: no. Three stories of opioids that have plagued Americans for more than 150 years.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Patrick McKenzie (patio11), patio11 (Patrick McKenzie):** Author's note: This is a copy of the white paper of the Working Group, a group of four pseudonymous professionals in Tokyo. I, Patrick McKenzie, was the primary author. It was distributed quietly during the week of March 25th, 2020. I have written an essay describing how this document came to be and demonstrating its provenance .

], [

**Throughline (NPR), Throughline (NPR):** Why does Whitney Houston's 1991 Super Bowl national anthem still resonate 30 years later? Listen to this episode from our friends at It's Been A Minute with Sam Sanders where they chat with author and Black Girl Songbook host Danyel Smith about that moment of Black history and what it says about race, patriotism and pop culture.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Throughline (NPR), Throughline (NPR):** Health insurance for millions of Americans is dependent on their jobs. But it's not like that everywhere. So how did the U. S. end up with such a fragile system that leaves so many vulnerable—or with no health insurance at all? On this episode, how a temporary solution created an everlasting problem.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Throughline (NPR), Throughline (NPR):** Is history always political? Who gets to decide? What happens when you challenge common narratives? In this episode, Throughline's Rund Abdelfatah and Ramtin Arablouei explore these questions with Nikole Hannah-Jones, an investigative journalist at the New York Times and the creator of the 1619 Project.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Throughline (NPR), Throughline (NPR):** When Andrew Johnson became president in 1865, the United States was in the midst of one of its most volatile chapters. The country was divided after fighting a bloody civil war and had just experienced the first presidential assassination. We look at how these factors led to the first presidential impeachment in American history.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Throughline (NPR), Throughline (NPR):** In the 2000 presidential election, results weren't known in one night, a week, or even a month. This week, we share an episode we loved from It's Been A Minute with Sam Sanders that revisits one of the most turbulent elections in U. S. history and what it could teach us as we wait for this election's outcome.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Throughline (NPR), Throughline (NPR):** NPR's new history podcast hosted by Ramtin Arablouei and Rund Abdelfatah. New episodes every Thursday starting February 7th.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Patrick McKenzie (patio11), patio11 (Patrick McKenzie):** A history of an anom-

aly in the 2020 Coronavirus Pandemic, an independent research project about it, and what we need to learn about our next steps.

], [

**Throughline (NPR), Throughline (NPR):** Have you ever wondered why your smartphone or toaster oven doesn't seem to last very long, even though technology is becoming better and better? In a special collaboration with Planet Money, we bring you the history of planned obsolescence – the idea that products are designed to break.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Throughline (NPR), Throughline (NPR):** The Sunni-Shia divide is a conflict that most people have heard about - two sects with Sunni Islam being in the majority and Shia Islam the minority. Exactly how did this conflict originate and when? We go through 1400 years of history to find the moment this divide first turned deadly and how it has evolved since.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], [

**Throughline (NPR), Throughline (NPR):** Zombies have become a global phenomenon — there have been at least ten zombie movies so far this year. Which made us wonder, where did this fascination for the undead come from? This week, how one of our favorite monsters is a window into Haiti's history and the horrors of slavery.

To manage podcast ad preferences, review the links below:

See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.

Learn more about sponsor message choices: podcastchoices.com/adchoices

NPR Privacy Policy

], ))

standard-article( title: [Bayard Rustin: The Man Behind the March on Washington], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Bayard Rustin, the man behind the March on Washington, was one of the most consequential architects of the civil rights movement you may never have heard of. Rustin imagined how nonviolent civil resistance could be used to dismantle segregation in the United States. He organized around the idea for years and eventually introduced it to Dr. Martin Luther King Jr. But his identity as a gay man made him a target, obscured his rightful status and made him feel forced to choose, again and again, which aspect of his identity was most important.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 133, edited-for-length: false, debug-mode: false, )

standard-article( title: [Make believe], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [Kids scream, “Monster in the hallway!”, and hide behind the couch. They stack up cushions for protection, and plan their defense. They know there’s not really a monster in the hallway, but it’s exciting to feel the adrenaline of panic, to make a shelter and feel safe.], [One kid yells, “The floor is hot lava!” Leaping between furniture is a fun challenge.], [One slips and wails, “Help! I’m falling! Save me! Save me!” Now one kid can feel protected, while the other gets to be the rescuing hero.], [Mom calls, “Pancakes are ready!”, and all stories stop when the kids run into the kitchen.], [Kids believe anything fun for a while. It’s called “make believe” because they’re making up beliefs.], [The game has a purpose. Each belief gives them a new situation, and lets them adopt a new role like protector or inventor.], [Grown-ups have their own version of make believe:], [“Everything happens for a reason.”], [“Those people are evil.”], [“I would be creatively prolific if I could quit my job.”], [None of these statements are true. But we like the way it feels to believe.], [Beliefs have a purpose.

They help us adopt a perspective or identity. They help us take action, or cooperate with others. The only problem is when we confuse belief with reality, and insist that something is absolutely true because we believe it.], [Beliefs don’t exist outside the mind. (Have you ever seen one in nature?) All beliefs are make believe.], ), insert-map: (:), word-count: 246, edited-for-length: false, debug-mode: false, )

*Each belief gives them a new situation, and lets them adopt a new role like protector or inventor.*

Derek Sivers

standard-article( title: [Daydreaming is my favorite pastime], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [Somewhere in our past, we were told it’s bad to daydream , because it meant doing nothing — staring out the window — instead of doing what we’re supposed to be doing. To admit we’re daydreaming felt like it needed an apology.], [But now I’ve finally embraced it. Deliberate daydreaming is my favorite pastime.], [About half the time that I used to read a book, I now just skip the book, and sit there daydreaming instead. And I almost never watch videos anymore. I just close my eyes and daydream.], [I find it most fun to ask myself a big question:], [Which were the top three best times in my life so far?], [What are my biggest regrets?], [What would I write a screenplay about?], [If I had the magic lamp, what would be my three wishes?], [What does the most ambitious version of myself look like?], [What about the least ambitious version of myself?], [How can I be a better dad?], [At what would I most love to become an expert?], [Is there anything I can’t do without?], [How would my life be different if I was blind? Deaf? Paralyzed?], [We’ve all had plenty of input.

standard-article( title: [Afghanistan: The Rise of the Taliban (2021)], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [How did a small group of Islamic students go from local vigilantes to one of the most infamous and enigmatic forces in the world? The Taliban is a name that has haunted the American imagination since 2001. The scenes of the group’s brutality repeatedly played in the Western media, while true, perhaps obscure our ability to see the complex origins of the Taliban and how they impact the lives of Afghans. It’s a shadow that reaches across the vast ancient Afghan homeland, the reputation of the modern state, and throughout global politics. At the end of the US war in Afghanistan we go back to the end of the Soviet Occupation and the start of the Afghan civil war to look at the rise of the Taliban.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices],

It's fun to let your mind direct its own entertainment.], ), insert-map: (:), word-count: 207, edited-for-length: false, debug-mode: false, )

[NPR Privacy Policy], ), insert-map: (:), word-count: 167, edited-for-length: false, debug-mode: false, )

*Deliberate daydreaming is my favorite pastime.*

Derek Sivers

standard-article( title: [We the People: Gun Rights], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [The Second Amendment. In April 1938, an Oklahoma bank robber was arrested for carrying an unregistered sawed-off shotgun across state lines. The robber, Jack Miller, put forward a novel defense: that a law banning him from carrying that gun violated his Second Amendment rights. For most of U. S. history, the Second Amendment was one of the sleepier ones. It rarely showed up in court, and was almost never used to challenge laws. Jack Miller's case changed that. And it set off a chain of events that would fundamentally change how U. S. law deals with guns. Today on Throughline's We the People: How the second amendment came out of the shadows. (Originally ran as The Right to Bear Arms )], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 159, edited-for-length: false, debug-mode: false, )

standard-article( title: [The Lord Of Misrule], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [On November 18, 1633, a book went to press in London. Its author, Thomas Morton, had been exiled from the Puritan colonies in Massachusetts for the crimes of drinking, carousing, and – crucially – building social and economic ties with Native people. Back in England, Morton wrote down his vision for what America could become. A very different vision than that of the Puritans.], [But the book wouldn't be published that day. It wouldn't be published for years. Because agents for the Puritan colonists stormed the press and destroyed every copy.], [Today on the show, the story of what's widely considered America's first banned book, the radical vision it conjured, and the man who outlined that vision: Thomas Morton, the Lord of Misrule.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 163, edited-for-length: false, debug-mode: false, )

standard-article( title: [It shows what you need to believe], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [In Harry Potter, there's a magic mirror that reflects the viewer's desire. What Harry sees in that mirror is very different than what Dumbledore or Ron sees, because their desires are all different.], [Imagine if there was something similar that shows you what you most need to believe right now . It shows proof to support whatever perspective would most benefit you. Upon seeing it, you instantly believe it, internalize it, and act upon it.], [Someone feeling sadly disconnected might see proof that everyone is connected.], [Someone trying to create something might see proof that people will love it.], [Someone feeling stuck by the seriousness of life might get un-stuck by the proof that our universe is actually a computer simulation.], [Someone with a terminal illness might see proof of a wonderful afterlife with loved ones waiting — to feel joy in their final days.], [We don't have to imagine this magic device. We already do this in real life. We find proof to support the perspective we need. Then we believe it.], [We don't have to argue what's in the magic mirror, which viewpoints are true or not , because everyone needs different beliefs for their different situations.], ), insert-map: (:), word-count: 199, edited-for-length: false, debug-mode: false, )

standard-article( title: [The Commentator], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Today the foundations of philosophy are seen as a straight line from Western antiquity, built on thinkers such as Plato and Aristotle. But, between the 8th century and 14th century, the West was greatly overshadowed by the Islamic world and philosophy was in very different hands. This week, how one Medieval Islamic philosopher put his pen to paper and shaped the modern world.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 103, edited-for-length: false, debug-mode: false, )

standard-article( title: [When Things Fall Apart (Throwback)], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Climate change, political unrest, random violence - Western society can often feel like what the filmmaker Werner Herzog calls,

standard-article( title: [By Accident of Birth], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [In August of 1895, a ship called the SS Coptic approached the coast of Northern California. On that boat was a passenger from San Francisco, a young

“a thin layer of ice on top of an ocean of chaos and darkness.” In the United States, polls indicate that many people believe that law and order is the only thing protecting us from the savagery of our neighbors, that the fundamental nature of humanity is competition and struggle. This idea is often called “veneer theory.” But is this idea rooted in historical reality? Is this actually what happens when societies face disasters? Are we always on the cusp of brutality?], [To access bonus episodes and listen to Throughline sponsor-free, subscribe to Throughline+ via Apple Podcasts or at plus.npr.org/throughline.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 159, edited-for-length: false, debug-mode: false, )

standard-article( title: [Palantir Will No Longer Profit Off of New Yorkers’ Health Data], author: [Sam Biddle], source-name: [The Intercept], images: (), paragraphs: ( [A controversial multimillion-dollar deal between New York City’s public hospital system and military contractor Palantir, first reported by The Intercept, is coming to an end, according to recent testimony before the city council.], [The Intercept reported in February that the New York City Health and Hospitals Corporation, which operates a network of public health care facilities across the city, had paid Palantir almost $4 million since 2023 for data analysis services. NYCHH says it used Palantir’s software to boost its efficiency in billing Medicaid and other public benefits, which included the automated scanning of patient health notes.], [The contract prompted protests from activists and local organizers who objected to the hospital system’s use of software from a company whose technology has facilitated lethal airstrike targeting, wide-reaching surveillance of American citizens, and deportation raids by Immigration and Customs Enforcement agents.], [“They should have no place in our hospitals, our pension funds, or our government.”], [At a March 16 meeting of the New York City Council, NYC Health + Hospitals CEO Mitchell Katz disclosed that Palantir’s contract will not be renewed come October. Katz defended the health care network’s collaboration with Palantir on the grounds that there was an “absolute firewall” between patient data and the company’s government customers, such as ICE, that would prevent information sharing. “We haven’t had any problems,” Katz said, “And we’re going to end the contract anyway because we always intended it to be a short-term solution.”], [According to Katz, data analysis previously conducted with Palantir’s help will be brought in-house following the contract’s expiration.], [“Palantir makes money by enabling mass violence in the U. S. and around the world. They should have no place in our hospitals, our

man named Wong Kim Ark who was returning home after visiting his wife and child in China. He’d taken trips like this before, and expected to come back to the city he was born in, to his life and friends. But when the ship docked, officials told him he couldn’t get off. The customs agent barred him according to the Chinese Exclusion Act, which denied citizenship to Chinese immigrants. Though Wong Kim Ark had been born in the U. S. and lived his whole life there, the agent said he was not a citizen.], [Wong was moved from steamer to steamer for months. But he was able to contact representatives from the Chinese Six Companies, a consortium of Chinese business owners that often hired legal representation for people subject to discrimination. His subsequent legal battles culminated in the 1897 Supreme Court case United States. v. Wong Kim Ark : a case that would forever change the path of American immigration law, and play a pivotal role in the ongoing battle over who gets to be a citizen of the United States.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 247, edited-for-length: false, debug-mode: false, )

standard-article( title: [Russia’s Vladimir Putin], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Vladimir Putin has been running Russia since 2000 when he was first elected as President. How did a former KGB officer make his way up to the top seat — was it political prowess or was he just the recipient of a lot of good fortune? In this episode, we dive into the life of Vladimir Putin and try to understand how he became Russia’s new “tsar.”], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 107, edited-for-length: false, debug-mode: false, )

pension funds, or our government," said Kenny Morris, an organizer with the American Friends Service Committee, which shared the contract documents with The Intercept.], ["Our campaign against Palantir doesn't stop in NYC," Morris said. "We will continue to isolate this company and limit its destructive influence on our lives. In this city and around the world, communities are organizing to push more and more corporate clients, institutions, and politicians to cut ties with Palantir."], [The post Palantir Will No Longer Profit Off of New Yorkers' Health Data appeared first on The Intercept .], ), insert-map: (:), word-count: 404, edited-for-length: false, debug-mode: false, )

*"We haven't had any problems," Katz said, "And we're going to end the contract anyway because we always intended it to be a short-term solution.*

Sam Biddle

standard-article( title: [A daily run and imagination], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [Every day you go for a run.], [Sometimes you feel yourself lagging, so you imagine there's a tiger on your tail, and that adrenaline gives you a turbo boost.], [Sometimes you feel like quitting, so you picture a pot of gold at the end, and that helps you finish.], [A running expert says you should act like you're running on hot coals, to keep you on the front of your feet. You try it, and it improves your stamina and energy.], [Sometimes, just for a change, you try running barefoot, or with your eyes closed, or with your arms out like an airplane. Every time you hear or think of a new way to run, you try it to see how it works and how you feel. The variety is fun.], [All you changed was the image in your mind, and that changed your emotions and actions.], [Ideas and beliefs are tools.

Choose them for the desired effect.], ), insert-map: (:), word-count: 158, edited-for-length: false, debug-mode: false, )

standard-article( title: [The Queen of Tupperware], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Who ushered housewives into the workforce and plastic storage containers into America's kitchens? Today on the show, the rise and fall of Brownie Wise, the woman behind Tupperware's plastic empire — and a revolution in women's work.], [Guests: Alison Clarke , author of Tupperware, the Promise of Plastic in 1950s America Bob Kealing , author of Life of the Party: The Remarkable Story of How Brownie Wise Built, and Lost, a Tupperware Party Empire], [To access bonus episodes and listen to Throughline sponsor-free, subscribe to Throughline+ via Apple Podcasts or at plus.npr.org/throughline .], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 133, edited-for-length: false, debug-mode: false, )

standard-article( title: [Judge the contents, not the box], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [My cousin took a course on a complete system for physical fitness and health, and followed every bit of its advice. She had great results at first. But then she saw the coach's social media posts, and hated his political beliefs, so now she doesn't follow that course at all.], [A best-selling book on psychology is filled with wisdom that would improve your life, if applied. But a few sentences were found to be plagiarized, or some of its studies don't replicate. So people trash the whole book and refuse to read it.], [That's the problem with judging a box instead of its contents. It's seeking "true" instead of useful. When any aspect of a package is flawed, it no longer feels "true", so all of it is discarded. You lose all of the benefits.], [Think of a famous person you despise, perhaps a politician or celebrity that represents everything wrong with the world. Now imagine hearing that person say something you really like. Hard to imagine, right? You've probably pre-decided that anything that comes out of that person's mouth is going to be bad. No matter what they say, you're against it, in advance. Judging someone as good or bad, instead of each individual idea

standard-article( title: [Doors and windows and what's real], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [Like everyone, I live in a little house with many doors and windows.], [One door goes out to my neighborhood. The local kids come to play with my dog. The elderly neighbors take so long to tell me their stories. I slow down my inner clock to listen.], [One window looks out at the nature around me. I'm getting to know this one tree really well. I toss a little dog food out there each day, and watch the local birds and rodents come by to eat it.], [One door is just for my son. This door goes somewhere new every time he opens it. I pause what I'm doing and follow him on an adventure. My inner clock stops working through that door.], [One door goes to my connections — the people around the world with mutual interests. A dozen people a day knock on this door and say hello. Sometimes more.], [One hidden door is for my dearest friends. That one comes all the way inside, anytime.], [One skylight looks far into the future. I daydream there a lot.], [One little locket looks at the past. I daydream there, too.], [But one door is really no fun to open. Whenever I do, I'm horrified at all the shouting. It's an infinite dark room filled with psychologically tortured

as useful or not.], [Listen to ideas, not their messenger. Focus on the contents, not the box. Avoid ideology.], [You've probably heard the phrase, "Perfect is the enemy of good." Likewise:

True is the enemy of useful.], ), insert-map: (:), word-count: 243, edited-for-length: false, debug-mode: false, )

standard-article( title: [Useful?], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [Let's define "useful" as whatever ultimately helps you do what you need to do, be who you want to be, or feel at peace .], [The word "ultimately" is there as a reminder of long-term consequences.], [Is it more useful to make others see your point of view, or make yourself see theirs?], [Is it more useful to think you're right, or to consider other perspectives?], [Is it more useful to make your life easier, or make yourself stronger?], [Is it ultimately more useful to benefit only yourself, or benefit others?], [It depends on who you want to be and what helps you feel at peace.], ), insert-map: (:), word-count: 106, edited-for-length: false, debug-mode: false, )

standard-article( title: [CodeSOD: Awaiting A Reaction], author: [Remy Porter], source-name: [The Daily WTF], images: (), paragraphs: ( [Today's Anonymous submitter sends us some React code. We'll look at the code and then talk about the WTF:], [/\/ inside a function for updating checkboxes on a page if (!e. target . checked ) { const removeIndex = await checkedlist. findIndex ( ( sel ) => sel. Id == selected. Id , ) const removeRowIndex = await RowValue . findIndex ( ( sel ) => sel == Index , )], [/\/ checkedlist and RowValue are both useState instances.... they should never be modified directly await checkedlist. splice (removeIndex, 1 ) await RowValue . splice (removeRowIndex, 1 )], [/\/ so instead of doing above logic in the set state, they dont setCheckedlist (checkedlist) setRow ( RowValue ) } else { if (checkedlist. findIndex ( ( sel ) => sel. Id == selected. Id ) == - 1 ) { await checkedlist. push (selected) } /\/ same, instead of just doing a set state call, we do awaits and self updates await RowValue . push ( Index ) setCheckedlist (checkedlist) setRow ( RowValue ) }], [Comments were added by our submitter.], [This code works. It's the wrong approach for doing things in React: modifying objects controlled by react, instead of using the provided methods, it's doing asynchronous push calls. Without the broader context, it's hard to point out all the other ways to do this, but honestly, that's not the interesting part.], [I'll let our submitter explain:], [This code is black magic, because if I update it, it breaks everything. Somehow, this is working in perfect tandem with the rest of the horrible page, but if I clean it up, it breaks the checkboxes; they're no longer able to be clicked. Its forcing React somehow to update asynchronously so it can use these updated values correctly, but thats the neat part, they people, trying to get attention. Strangers screaming at strangers, starting fights. Businesses put windows there, showing bad things said and done today, because they make money when people get mad.], [They say I'm supposed to open that door, because that's the real world.], [But it seems a lot less real than what's in the other doors and windows in my life.], ), insert-map: (:), word-count: 289, edited-for-length: false, debug-mode: false, )

standard-article( title: [Octavia Butler: Visionary Fiction (2021)], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Octavia Butler's alternate realities and 'speculative fiction' reveal striking, and often devastating parallels to the world we live in today. She was a deep observer of the human condition, perplexed and inspired by our propensity towards self-destruction. Butler was also fascinated by the cyclical nature of history, and often looked to the past when writing about the future. Along with her warning is her message of hope - a hope conjured by centuries of survival and persistence. For every society that perishes in her books comes a story of rebuilding, of repair.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 132, edited-for-length: false, debug-mode: false, )

standard-article( title: [The Modern White Power Movement (2020)], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [The recent shooting in Buffalo, New York, which authorities are investigating as a hate crime, has yet again highlighted the threat posed by domestic terrorism in the U. S. At the center are violent extremists – the most lethal and persistent of whom are white supremacists and anti-government militias. They're part of a deeply interconnected movement which, since the 1980s, has pursued a mission to topple the U. S government with guerrilla warfare. Today, this movement is made up of highly-organized groups with paramilitary capabilities, but it hasn't always been this way. This week, we trace the rise of the modern white power movement.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 142, edited-for-length: false, debug-mode: false, )

aren't even being used anywhere else, but somehow the re-rendering page only accepts awaits. I've tried refactoring it 5 different ways to no avail], [That's what makes truly bad code. Code so bad that you can't even fix it without breaking a thousand other things. Code that you have to carefully, slowly, pick through and gently refactor, discovering all sorts of random side-effects that are hidden. The code so bad that you actually have to live with it, at least for awhile.], [[Advertisement] BuildMaster allows you to create a self-service release management platform that allows different teams to manage their applications. Explore how!], [style="clear: left;">], ), insert-map: (:), word-count: 406, edited-for-length: false, debug-mode: false, )

standard-article( title: [The lasting legacy of the slave patrols], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [To this day, America continues to grapple with the legacy of slavery. On this week's episode, we explore the creation of slave patrols, which were created to control the movement of enslaved Black people in the 1700s, and how those patrols shaped American society and modern policing. To access bonus episodes and listen to Throughline sponsor-free, subscribe to Throughline+ via Apple Podcasts or at plus.npr.org/throughline .], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 106, edited-for-length: false, debug-mode: false, )

standard-article( title: [Considerate book pricing], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [I love having my own store so I can make things the way I think they should be.], [For example, I disagree with the usual pricing of books. If I buy a book in one format, it doesn't seem fair to pay full price to get it in another format. That would be paying twice for the same content.], [Let's separate these two things:], [Contents : the words in a book], [Delivery : the ways to get the words into your brain: paper, audio, PDF, HTML, etc.], [What we really want is to buy the contents, not delivery .], [With so many different devices now, it seems fair that if you buy the contents of a book, it should include all formats of delivery. EPUB, MP3, Kindle, M4B, PDF, HTML, or whatever new formats may come in the future.], [Today you want to read silently by the fire. Tomorrow you want to listen while you drive. In ten years, you want to read it again on your new device.

This should all be included when you buy a book.], [I love this idea. It's almost perfect. It has just one problem:

Paper costs money.

So I can't just include it for free.], [But following this philosophy, it's not right to charge full price for each paper book, because that would mean paying repeatedly for the contents!], [So here's the solution I came up with:], [Contents of my book: $15], [Delivery of all digital formats: FREE], [Paper? Just cover its cost: $4 each (+ postage)], [I like this.

It means you never pay for the contents twice.

It works out well for many different scenarios:], [All digital formats: $15], [First paper book: $19], [Paper books after you've bought the contents: $4 each], [Loved the ebook, and now want to buy 20 paper copies for your friends? Just $4 each, so $80 total for 20 hardcover books .], [It's worked out well. People are buying many hardcover copies as gifts. A few people have bought over 500 copies each to give to clients or members of their organization.], [The only downside of creative pricing is it requires a little explanation. Like anything unusual.], [Log in to sivers.com to get my books. ☺], [See my previous article on pricing philosophy for more thoughts on creative pricing .], [Read " Make a dream come true " for more thoughts on making things the way they should be .], ), insert-map: (:), word-count: 404, edited-for-length: false, debug-mode: false, )

standard-article( title: [The Dance of the Dead (2021)], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Long before it was a sugary moviefest, the Halloween we know was called Samhain. The Celts of ancient Ireland believed Samhain was a night when the barrier between worlds was thin, the dead could cross over, and if you didn't disguise yourself, evil fairies might spirit you away. Over time the holiday shape-shifted too, thanks to the Catholic Church, pagan groups, and even the brewing company Coors. From the Great Famine of Ireland to Elvira and the Simpsons, we present the many faces of Halloween.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 125, edited-for-length: false, debug-mode: false, )

standard-article( title: [Anything You Want — third edition for 2022], author: [Derek Sivers], source-name: [Derek Sivers], images: (), paragraphs: ( [In 2011, I got a phone call from a number I didn't recognize.], ["Hello?"], ["Derek. It's Seth Godin."], ["Wow! Hi Seth!"], ["I'm starting a new publishing company, so I want you to write a book. Short, like a manifesto. Will you do it?"], ["Uh, sure!"], ["Great. I look forward to it."], ["Thanks!"], [Over the next eleven days, I wrote the lessons I'd learned from starting, growing, and selling my company. He liked it, named it " Anything You Want ", and a few weeks later it was for sale. My first book. Simple as that.], [To me, it was no big deal - just telling my tale . But a lot of small business owners said my book helped them remember their purpose, and get re-excited about their business.], [Penguin bought Seth's publishing company and re-released my book on Penguin Portfolio. But I wanted to self-publish.], [I want to own the rights to my books so I can do what I want with them: give them away, let people translate them , or sell them how I like. I'm thinking long-term. I've got many books to come, and I want them to be a matching set.

So I bought back the rights from Penguin.], [I improved many chapters in "Anything You Want", and added eight new chapters that were missing — points that people kept asking about over my last ten years of talking about this book — better explanations — better stories.], [Then I gave it a new cover to match all of my other books. (White, to represent its innocent naïveté.) I recorded the new audiobook , better than before.], [And now, it's finally available, in its third and final edition, directly from me .], [" ANYTHING YOU WANT: 40 lessons for a new kind of entrepreneur "], [Get it at sivers.com .], [There are only 5000 limited-edition linen hardcovers printed, so get that version while they last. Even if you already read it, consider getting this as the permanent edition.], [The first copy is $19, which is $15 for the content, and $4 for the paper itself .

All future copies cost only the price of the paper: $4.

Paper books include all digital formats for free. As before, I'm giving all profits to charity .], ), insert-map: (:), word-count: 379, edited-for-length: false, debug-mode: false, )

*) I recorded the new audiobook , better than before.*

Derek Sivers

standard-article( title: [The Hidden War], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [How does a country go from its leader winning the Nobel Peace Prize to all-out war in just one year? That's the question surrounding Ethiopia, which has become embroiled in one of the deadliest wars of the 21st century. The U. S. has called it an ethnic cleansing campaign against Tigrayans, a minority group in the country; some human rights organizations have called it a genocide. But many people outside Ethiopia and its diaspora had no idea it was happening. In U. S. media, it's hardly discussed, even as violence has intensified throughout the country. In this episode, we tell the story of Ethiopia — the oldest independent country in Africa — and the political,

standard-article( title: [Before Roe: The Physicians' Crusade], author: [Throughline (NPR)], source-name: [Throughline (NPR)], images: (), paragraphs: ( [Abortion wasn't always controversial. In fact, in colonial America it would have been considered a fairly common practice: a private decision made by women, and aided mostly by midwives. But in the mid-1800s, a small group of physicians set out to change that. Obstetrics was a new field, and they wanted it to be their domain—meaning, the domain of men and medicine. Led by a zealous young doctor named Horatio Storer, they launched a campaign to make abortion illegal in every state, spreading a potent cloud of moral righteousness and racial panic that one historian later called "the physicians' crusade." And so began the

cultural and religious factors that led to this war.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 162, edited-for-length: false, debug-mode: false, )

century of criminalization.], [In the first episode of a two-part series, we're telling the story of that century: how doctors put themselves at the center of legal battles over abortion, first to criminalize — and then to legalize.], [To manage podcast ad preferences, review the links below:], [See pcm.adswizz.com for information about our collection and use of personal data for sponsorship and to manage your podcast sponsorship preferences.], [Learn more about sponsor message choices: podcastchoices.com/adchoices], [NPR Privacy Policy], ), insert-map: (:), word-count: 182, edited-for-length: false, debug-mode: false, )

**The Urban Monitor**

Vol. 1, No. 087 · 2026-03-30

*Curated and composed automatically.*